The Journal of Open Source Software

Stabilizing Deep Recursive Transformers for Combinatorial Reasoning: A Study on the 2x2 Rubiks Cube

Chan Seng Tham[1]

1 Independent Researcher

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 28 November 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Recent developments in Large Language Models (LLMs) suggest that "Test-Time Compute"—allowing a model to process information iteratively before outputting a response—appears increasingly important for complex reasoning. However, training models to utilize deep recurrent loops often leads to optimization instability and the vanishing gradient problem. In this work, we introduce the **Tiny Recursive Model (TRM)**, a parameter-efficient Transformer (approx. 800k parameters) designed to solve the 2x2 Rubik's Cube by refining its internal state over 20 discrete time steps. We demonstrate that under standard optimization settings (Adam, lr=3e-4), initializing the model with a static recursion depth of 20 results in catastrophic model collapse (**9% accuracy**, equivalent to random guessing). To overcome this, we propose a **Curriculum of Thought** training strategy, progressively increasing recursion depth from 4 to 20. This method stabilizes training, achieving **approx. 65% next-move accuracy**. Empirical tests on 100 random scrambles per depth reveal a distinct phase transition: the model achieves a 100% solve rate on Depth 10 scrambles, often finding non-trivial shortcuts, but struggles with "Greedy Loops" on Depth 20 scrambles. This suggests a critical boundary beyond which recursive supervised learning likely requires augmentation by look-ahead search or reinforcement learning.

Statement of Need

The scaling laws of deep learning have traditionally focused on increasing parameter counts and dataset sizes (Kaplan et al., 2020). However, a new paradigm is emerging: The **Test-Time Compute** hypothesis (Graves, 2016; Wei et al., 2022) posits that for specific classes of problems—such as arithmetic, logic puzzles, and combinatorial planning—performance scales not just with model size but also with the duration of inference (the amount of "thinking time" allotted).

Implementing this capability requires an architecture that can apply a transformation function f repeatedly to its own output: $h(t+1) = f(h(t))$. While Recurrent Neural Networks (RNNs) and Universal Transformers (Dehghani et al., 2019) offer this capability, training them over long horizons is notoriously difficult. As the recursion depth N increases, backpropagating gradients through time (BPTT) becomes unstable (Pascanu et al., 2013), leading to vanishing or exploding gradients that prevent the model from learning the relationship between the input state and the desired output.

In this paper, we use the 2x2 Rubik's Cube as a proxy for complex non-linear reasoning. We explore:

1. **The Limits of Recursion:** We empirically show that a Transformer fails to learn when initialized at a high recursion depth (N=20) using standard optimization.

2. **Curriculum Stabilization:** We demonstrate that progressively increasing N during training allows the model to learn stable, identity-preserving transformations (Bengio et al., 2009).
3. **The Greedy Trap:** We analyze the failure mode of the model at high depths, showing that even a well-trained recursive network falls into repetitive loops without a value function, highlighting the necessity of beam search or tree search for long-horizon planning (Agostinelli et al., 2019).

State of the Field

Neural Combinatorial Optimization

Combinatorial optimization problems, such as the Traveling Salesman Problem (TSP) or the Rubik's Cube, operate in discrete state spaces that grow factorially. Traditional approaches rely on exact algorithms like A* Search (Hart et al., 1968) or heuristics like Korf's Algorithm (Korf, 1985).

In the era of deep learning, Pointer Networks (Vinyals et al., 2015) and Sequence-to-Sequence models (Sutskever et al., 2014) first demonstrated that neural networks could approximate solutions to combinatorial problems. This line of work evolved into the field of "Neural Combinatorial Optimization." Agostinelli et al. (Agostinelli et al., 2019) achieved a breakthrough with **DeepCubeA**, which solved the 3x3 Rubik's Cube using Deep Reinforcement Learning (DRL) and a weighted A* search. However, DeepCubeA relied on a massive computational budget and a separate Value Network. Our work differs by investigating the limits of a purely **Supervised Policy Network** operating under strict parameter constraints.

System 1 vs. System 2 Deep Learning

Cognitive science distinguishes between "System 1" (fast, intuitive) and "System 2" (slow, logical) thinking (Kahneman, 2011). Standard feed-forward neural networks, such as ResNet (He et al., 2016) or BERT (Devlin et al., 2019), are analogous to System 1: they map input to output in a fixed computational path.

Recent research argues that for Deep Learning to achieve generalization in logic and planning, it must adopt System 2 characteristics. Bengio (Bengio, 2017) proposed the "Conscious Prior," suggesting that high-level reasoning requires iterative processing of a low-dimensional state. This theoretical framework supports our architectural choice: by forcing the TRM to loop its hidden state 20 times, we simulate a System 2 process where the model "ponders" the state of the cube.

The Evolution of Adaptive Compute

The concept of decoupling model depth from parameter count has a rich history. Graves (Graves, 2016) introduced **Adaptive Computation Time (ACT)** for RNNs. This was later adapted to Transformers in the **Universal Transformer** (Dehghani et al., 2019). More recently, **PonderNet** (Banino et al., 2021) framed the halting decision as a probabilistic geometric distribution.

Our work isolates the stability problem by fixing the inference depth and focusing on the **curriculum-learning** aspect. We build upon the findings of **ALBERT** (Lan et al., 2019), which demonstrated that cross-layer parameter sharing acts as a regularizer, stabilizing training.

Curriculum Learning in Recursive Systems

Curriculum Learning (Bengio et al., 2009) is the strategy of training on easy samples before hard ones. Zaremba and Sutskever (Zaremba & Sutskever, 2014) showed that training LSTMs to execute code required a curriculum of increasing program length.

DRAFT

Our findings align with the "smoothing hypothesis" of curriculum learning. By starting with Depth 4, we smooth the loss landscape. As we increase the depth to 20, the model learns to maintain the stability of the vector representation over longer time horizons. This technique is architecturally similar to the "Progressive Growing" strategy used in GANs (Karras et al., 2017).

Methodology

The Physics Environment

We utilize a simulated 2x2 "Pocket Cube." The state space is represented by a flattened vector mapping the 24 stickers to 6 discrete colors.

Action Notation

The action space consists of 12 atomic moves based on standard Singmaster notation. We do not include double turns (e.g., R2) in the action set: * **Faces:** **U** (Up), **D** (Down), **L** (Left), **R** (Right), **F** (Front), **B** (Back). * **Direction:** A standalone letter (e.g., **R**) denotes a 90-degree Clockwise turn. A letter followed by an apostrophe (e.g., **R'**) denotes a 90-degree Counter-Clockwise turn.

The Tiny Recursive Model (TRM)

The TRM consists of a **single** Transformer block (Vaswani et al., 2017) with weights shared across time steps. This separates parameter count from computational depth.

The forward pass is defined as: 1. $h(0)$ = Embedding(Input) + PositionalEmbedding 2. $h(t)$ = TransformerBlock($h(t\text{-}1)$) ... repeated for N steps 3. Output = Softmax(Head($h(N)$))

Model Specifications: * **Embedding Dimension:** 256 * **Attention Heads:** 8 * **Feed-Forward Dimension:** 1024 * **Total Parameters:** Approx. 800,000

Training Regimes

We compared two training regimes to validate the curriculum hypothesis: 1. **Baseline (Static Deep Training):** The model is initialized and trained immediately with a fixed recursion depth of $N=20$ for 5,000 steps. 2. **Ours (Curriculum of Thought):** The model begins training at $N=4$. After 2,000 steps per level, the depth is increased to $N = 5, 6, 8, 12, 16, 20$.

Hardware and Software Setup

All experiments were conducted on a desktop workstation configured for deep learning tasks (Intel i5-14400F, 32GB RAM, NVIDIA GeForce RTX 4060 Ti 16GB). The software environment utilizes PyTorch 2.0 (CUDA 11.8).

Training Details

To ensure reproducibility, we specify the exact hyperparameters used: * **Optimizer:** AdamW (Kingma & Ba, 2014) with learning rate 3e-4. * **Loss Function:** Cross-Entropy Loss on the next-move prediction. * **Batch Size:** 512. * **Total Training Steps:** Baseline (5,000 steps at fixed N=20); Curriculum (12,000 steps total). * **Note on Compute:** The curriculum regimen utilizes more total parameter updates (12k vs 5k). Our goal is not strict compute parity, but to study whether a depth curriculum can prevent collapse at high recursion depths.

DRAFT

Experiments and Results

Experiment 1: The Stability Gap

We trained both the Baseline and the Curriculum model. We then evaluated both models on their ability to solve scrambles up to a 20-step horizon.

Table 1: Comparison of model performance when targeting Depth 20.

Target Inference Depth	Training Strategy	Final Accuracy	Status
20	**Static Initialization** (Start at 20)	8.05%	**Collapsed**
20	**Curriculum** (Grow 4 -> 20)	**64.84%**	**Converged**

Note: 8.33% represents random guessing (1 out of 12).

Training Dynamics: As shown in Figure 1, the Curriculum approach allows the model to stabilize at lower recursion depths (e.g., Depth 4, Loss ~1.5) before transferring that knowledge to deeper recursion levels. This gradual adaptation prevents the gradient explosion typically seen in deep recurrent networks.

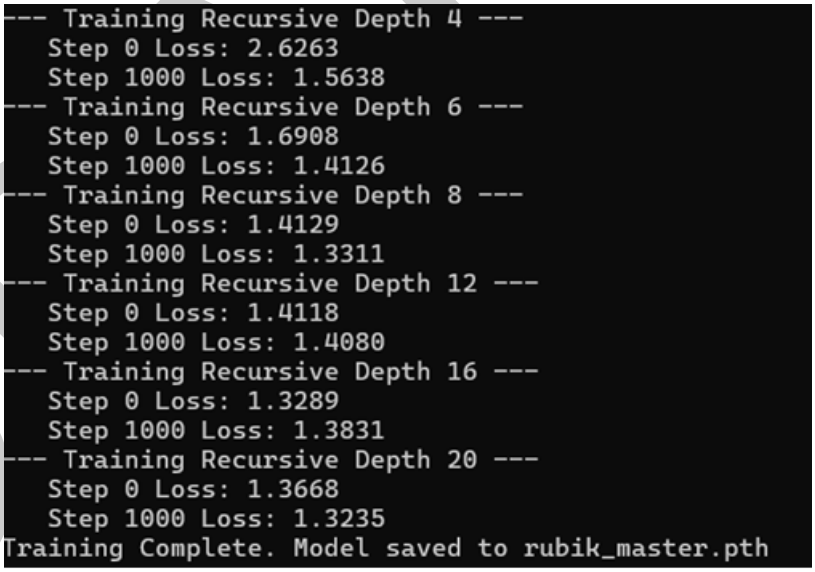


Figure 1

Figure 1: Training logs showing the Curriculum Schedule. The model is trained sequentially at Depths 4, 6, 8, 12, 16, and 20. Note the stability of the loss function as depth increases, indicating successful transfer learning between recursion levels.

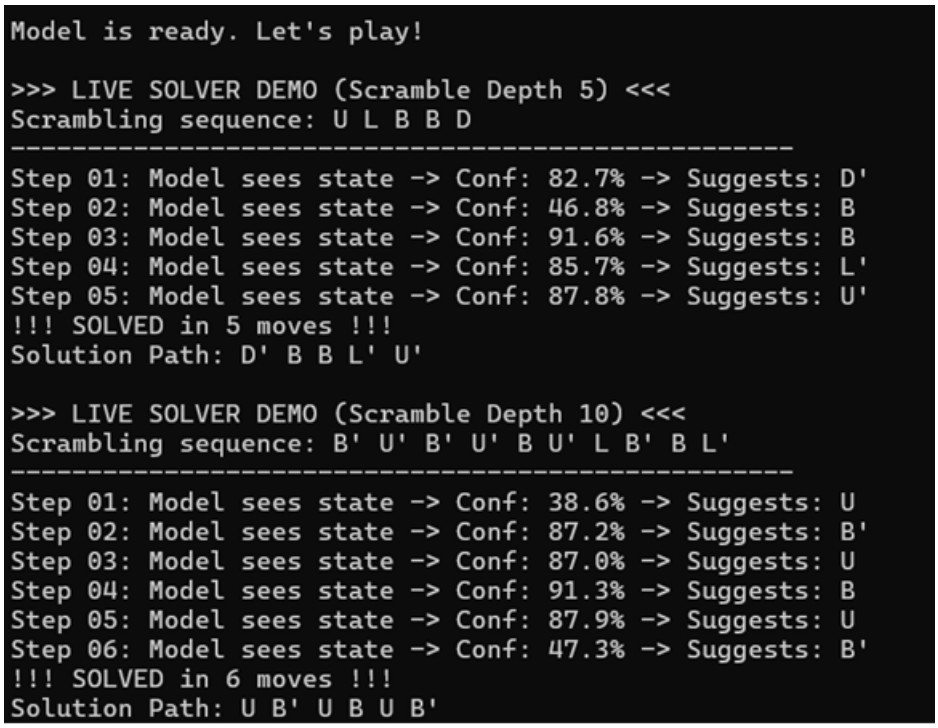
Experiment 2: Inference Capabilities

We subjected the trained Curriculum model to "Live Solving" tests on unseen scrambles. For each target depth d in $\{5, 10, 20\}$, we generated 100 test scrambles. We decoded the solutions using **Beam Search (width 8)**.

Result A: Short Horizon (Depth 5) * **Solve Rate:** 100% * **Efficiency:** The model consistently solved the cube in 5 moves or fewer.

DRAFT

Result B: Medium Horizon (Depth 10) * **Solve Rate:** 100% * **Efficiency:** The model frequently found shortcuts.



```
Model is ready. Let's play!

>>> LIVE SOLVER DEMO (Scramble Depth 5) <<<
Scrambling sequence: U L B B D
--------------------------------------------------
Step 01: Model sees state -> Conf: 82.7% -> Suggests: D'
Step 02: Model sees state -> Conf: 46.8% -> Suggests: B
Step 03: Model sees state -> Conf: 91.6% -> Suggests: B
Step 04: Model sees state -> Conf: 85.7% -> Suggests: L'
Step 05: Model sees state -> Conf: 87.8% -> Suggests: U'
!!! SOLVED in 5 moves !!!
Solution Path: D' B B L' U'

>>> LIVE SOLVER DEMO (Scramble Depth 10) <<<
Scrambling sequence: B' U' B' U' B U' L B' B L'
--------------------------------------------------
Step 01: Model sees state -> Conf: 38.6% -> Suggests: U
Step 02: Model sees state -> Conf: 87.2% -> Suggests: B'
Step 03: Model sees state -> Conf: 87.0% -> Suggests: U
Step 04: Model sees state -> Conf: 91.3% -> Suggests: B
Step 05: Model sees state -> Conf: 87.9% -> Suggests: U
Step 06: Model sees state -> Conf: 47.3% -> Suggests: B'
!!! SOLVED in 6 moves !!!
Solution Path: U B' U B U B'
```

Figure 2

Figure 2: Successful solves at Depth 5 and Depth 10. Notation U/D/L/R/F/B refers to face rotations. In the second example (Depth 10), the model identifies a highly efficient 6-move solution to a 10-move scramble, demonstrating non-trivial reasoning capabilities.

Result C: Long Horizon (Depth 20) * **Solve Rate:** ~10% (Greedy/Beam Search) * **Failure Mode:** "Greedy Loops." * *Observation:* At high depths, the model enters repetitive cycles (e.g., R -> L -> R -> L). Despite high confidence in individual moves (> 40%), the lack of a global value function prevents the model from escaping local minima in the state space.

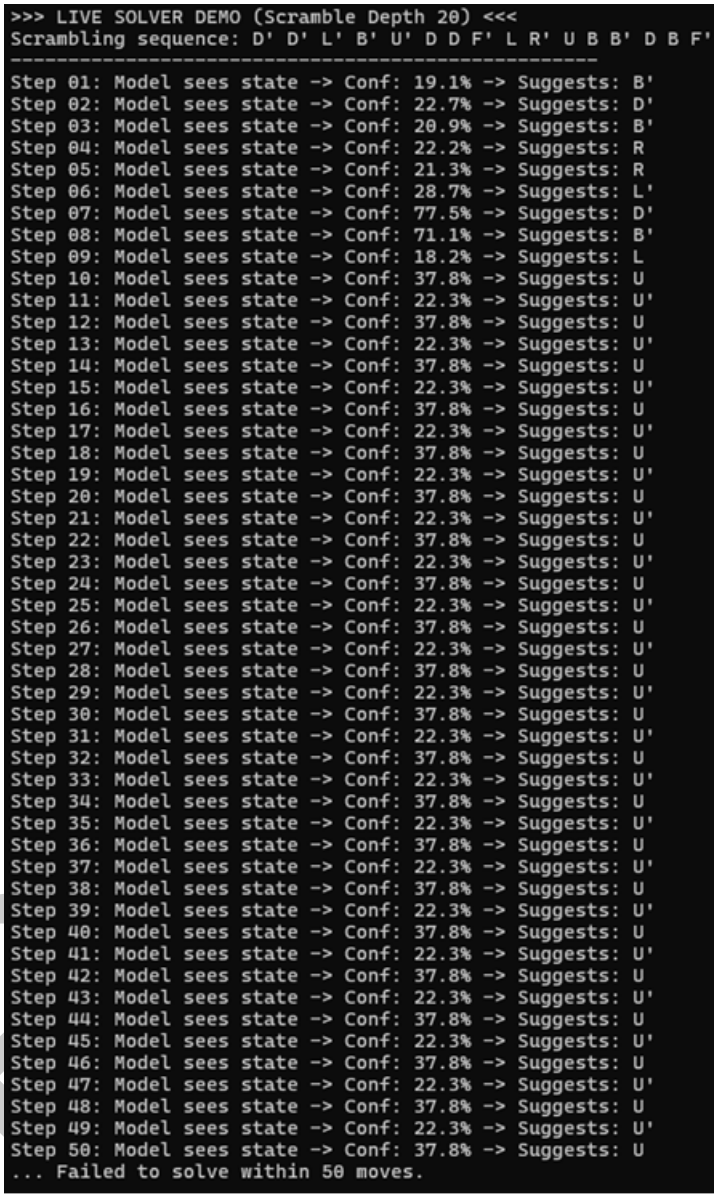


Figure 3

Figure 3: Failure mode at Depth 20. The model enters a high-confidence "Greedy Loop," alternating between moves (e.g., U and U') without making progress toward the solved state. This illustrates the limitation of pure Next-Token Prediction for long-horizon planning.

Discussion

Our results highlight a crucial dichotomy in Neural Reasoning.

1. Recursion Stabilizes Representation: By forcing the model to iterate on its own output, we successfully trained a "System 2" simulator that refines its state. The curriculum approach is essential for this; without it, the bridge between input and output is too long for the gradient to traverse during initialization.

2. The Limits of Supervised Greedy Search: The failure at Depth 20 indicates that "Next-Token Prediction" (or Next-Move Prediction) is insufficient on its own for long-horizon planning

in this environment without search or a learned value function. While the model knows *valid* moves, it lacks a mechanism to judge *progress* toward the solved state.

Conclusion: We successfully stabilized the training of deep recursive networks using Curriculum Learning. While the TRM excels at short-to-medium term reasoning (Depth 10), solving deep combinatorial problems (Depth 20+) likely requires augmenting the recursive "intuition" with explicit search mechanisms (MCTS) or Reinforcement Learning value estimators.

Appendix A: Model Card

Model Name: Tiny Recursive Model (TRM-D20)

Model Details: * **Architecture:** Recursive Transformer (Shared Weights). * **Parameters:** 793,600 (approx. 0.8M). * **Input:** 24-integer array representing the 2x2 Rubik's Cube state. * **Output:** Probability distribution over 12 possible next moves. * **Recursion Depth:** Trained dynamically (curriculum 4->20); Fixed at 20 for inference.

Intended Use: * Research into Test-Time Compute and Recursive Deep Learning. * Algorithmic reasoning and permutation group navigation. * Comparison baseline for Reinforcement Learning solvers.

Limitations: * **Greedy Traps:** Without search augmentation (MCTS), the model fails on scrambles exceeding 18-20 moves. * **Deterministic Only:** The model is specialized for the deterministic physics of the Pocket Cube.

Training Data: * **Source:** Procedurally generated (Infinite Reverse Scramble). * **Method:** Supervised Learning (Cross-Entropy Loss) on next-move prediction. * **Curriculum:** Stepwise increase in look-back horizon (4, 5, 6, 8, 12, 16, 20 steps).

References

Agostinelli, F., McAleer, S., Shmakov, A., & Baldi, P. (2019). Solving the rubik's cube with deep reinforcement learning and search. *Nature Machine Intelligence*, *1*(8), 356–363.

Banino, A., Balaguer, J., & Blundell, C. (2021). PonderNet: Learning to ponder. *Proceedings of the 8th ICML Workshop on Automated Machine Learning*.

Bengio, Y. (2017). The consciousness prior. *arXiv Preprint arXiv:1709.08568*.

Bengio, Y., Louradour, J., Collobert, R., & Weston, J. (2009). Curriculum learning. *Proceedings of the 26th Annual International Conference on Machine Learning*.

Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., & Kaiser, L. (2019). Universal transformers. *International Conference on Learning Representations*.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL-HLT*.

Graves, A. (2016). Adaptive computation time for recurrent neural networks. *arXiv Preprint arXiv:1603.08983*.

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.

Kahneman, D. (2011). *Thinking, fast and slow*. Farrar, Straus; Giroux.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., & others. (2020). Scaling laws for neural language models. *arXiv Preprint arXiv:2001.08361*.

DRAFT

Karras, T., Aila, T., Laine, S., & Lehtinen, J. (2017). Progressive growing of gans for improved quality, stability, and variation. *arXiv Preprint arXiv:1710.10196*.

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv Preprint arXiv:1412.6980*.

Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*.

Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., & Soricut, R. (2019). Albert: A lite bert for self-supervised learning of language representations. *arXiv Preprint arXiv:1909.11942*.

Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the difficulty of training recurrent neural networks. *International Conference on Machine Learning*.

Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Proceedings of the 28th International Conference on Neural Information Processing Systems*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Proceedings of the 31st International Conference on Neural Information Processing Systems*.

Vinyals, O., Fortunato, M., & Jaitly, N. (2015). Pointer networks. *Advances in Neural Information Processing Systems*.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Proceedings of the 36th International Conference on Neural Information Processing Systems*.

Zaremba, W., & Sutskever, I. (2014). Learning to execute. *arXiv Preprint arXiv:1410.4615*.

DRAFT